

Project Report

Project Name: Purrdict

Reference ID: DCG9D1RC

1. Executive Summary

Purrdict is an IoT cat-monitoring platform that pairs an ESP32-CAM smart collar/leash with a Next.js web app (also shipped as an Android app via Capacitor) to track a cat's behavior, emotional state, location, and health. Unlike generic "AI" pet trackers, every detection rule in Purrdict is traceable to one of 17 peer-reviewed veterinary and ethology papers, turning a sensor stream into evidence-based, cited insights rather than a black box. The result is a dashboard that tells an owner what their cat is doing right now, why the system believes that, and whether it's something to worry about.

2. Project Overview

2a. Why we're building this

Most consumer pet trackers report raw activity counts or make vague "AI-powered" behavior claims with no way to verify them. Cat owners are left guessing whether a change in activity means something is actually wrong. Purrdict addresses that gap by mapping observable sensor signatures (motion intensity, frame-to-frame change, activity timing) directly onto behavior and welfare categories defined in published research — so every reading on the dashboard cites the paper and method behind it (e.g. Ikurior et al. 2023's ODBA thresholds, Evangelista et al. 2019's Feline Grimace Scale, Nicholson & O'Carroll 2021's feline emotion ethogram).

2b. How it relates to the theme

The hackathon theme, "Coding for Kitties," is the core of the product rather than a surface-level tie-in: the entire system — hardware, backend, and UI — exists to answer one question for a cat owner: *"What is my cat up to, and is she okay?"* Every feature (live behavior feed, emotion assessment, pain/wellness check, GPS geofencing, needs prediction, scrapbook, reminders) is built around cat welfare and care.

2c. Target Audience

Cat owners who want more insight into their indoor/outdoor cat's daily behavior and wellbeing than a standard activity tracker provides — particularly owners who want a leash-based device that also gives them a live camera view, GPS location, and evidence-based health signals, without needing to trust an opaque AI model.

3. Key Features

Auth, Onboarding & Cat Profiles

- ✓ Signup / login with JWT sessions and bcrypt (cost 12) password hashing (`/api/auth/signup` , `/api/auth/login`)
- ✓ Sign out (clears auth state, localStorage, and JWT)
- ✓ 4-step guided setup wizard: ESP32 IP pairing → name the cat → profile details (breed, age, fur color, icon or photo upload) → trading-card style preview

- ☒ Cat cards on the dashboard with edit (name/breed/age/photo/fur color) and delete, full CRUD via `/api/cats` and `/api/cats/[id]`
- ☒ Support for multiple cats per account ("Add another cat")
- ☒ One-click Demo Mode — instantly bootstraps a sample cat ("Whiskers") so judges/users can try the app without hardware
- ☒ First-run guided tutorial overlay walking new users through Live, Cam, Map, Tasks, and Album tabs

Live Behavior & Wellness Detection

- ☒ Live behavior detection feed — 10 behaviors (sleeping, grooming, eating, playing, walking, sitting/alert, scratching, running, drinking, jumping) classified from ESP32 motion data, each shown with confidence % and its research citation
- ☒ Emotion assessment — 5 states (contentment, excitement, relaxation, curiosity, alertness) derived from motion-behavior correlates (Nicholson & O'Carroll 2021)
- ☒ Wellness / pain check panel citing the Evangelista et al. 2023 pain ethogram
- ☒ Circadian rhythm context card showing time-of-day activity phase and a 24-hour activity pattern chart (Piccione et al. 2013 crepuscular framework)
- ☒ Observation history timeline polling live sensor/behavior data from the database every 10 seconds
- ☒ Research Basis panel citing all 17 supporting papers
- ☒ *Current limitation:* the Emotion Assessment panel is a client-side lookup keyed off the currently-displayed behavior label (`deriveEmotion()` in `dashboard/page.tsx`), not a database-backed emotion pipeline — the `emotion_assessments` table and its logging activity exist in the schema/Temporal code but nothing in the app currently writes to it. The Wellness Check panel similarly always renders a static "Score: 0/10 — No Anomalies" — it is not computed from any 14-day baseline yet.

Live Cat Cam

- ☒ Live MJPEG video stream from the ESP32-CAM, proxied server-side (`/api/esp32/stream`) with basic SSRF protection (only private/local IP ranges accepted)
- ☒ Snapshot capture — grabs the current video frame to canvas and downloads it as a JPEG
- ☒ Auto-reconnect / force-reconnect handling for the ESP32's single-viewer stream limitation
- ☒ Settings modal to configure and persist the ESP32's IP address
- ☒ Snapshot fallback endpoint (`/api/esp32/snapshot`) so the dashboard has a still image even when the live stream isn't open

GPS Tracking — "Away From Home" Detection & Distance

- ☒ **Home setup:** the owner types their home address once; it's geocoded to lat/lng via OpenStreetMap Nominatim and saved to `localStorage`, giving the geofence a fixed center point to measure against.
- ☒ **Live map:** a real Leaflet.js + OpenStreetMap map renders the home point, a dashed geofence circle (80m radius), a pulsing cat marker for the current position, and a yellow dashed trail line connecting recent positions.
- ☒ **Distance-from-home calculation:** on every position update, the app converts the lat/lng delta between the cat's current point and home into meters using an equirectangular approximation ($\Delta lat \times 111,320m$ and $\Delta lng \times 111,320m \times \cos(homeLat)$), then takes the Euclidean distance — this is what powers the live "Xm from home" readout and the distance progress bar.
- ☒ **Away-from-home alert:** that live distance is compared against the 80m geofence radius. Inside the radius shows a green "● SAFE ZONE" badge; the moment the cat's position crosses outside it, the badge flips to a red "⚠ OUTSIDE" warning and the distance bar turns red — an immediate visual alert that the cat has wandered off.

- ☒ **Backend workflow exists but also simulated:** a Temporal workflow (`gpsMonitoringWorkflow` , triggered via `POST /api/gps/monitor`) logs GPS coordinates and runs the same haversine geofence check server-side. Its retry/durability behavior is real, but the workflow itself generates coordinates with `Math.random()` around the home point rather than reading from actual ESP32 hardware — there is no GPS module in the current firmware, so neither the frontend nor the backend workflow has a real coordinate source to plug into yet.
- ☒ *Current limitation:* the position feed on the map page is a simulated wandering path around the home point (cycled every 3s), matching the same simulate-and-log pattern in the backend workflow — the geofence math, distance display, and safe/outside alert states are fully real, but both layers are waiting on an actual GPS sensor.

Vet Finder — Search & "Booking" Flow

- ☒ **Search by location:** the owner types a city/country (or taps "Use my current location," which calls the browser Geolocation API and reverse-geocodes it via Nominatim) to pull up nearby veterinary clinics — name, address, phone, hours, and star rating.
- ☒ **Real distance ranking:** once a home address is set, every clinic's straight-line distance from home is computed with the haversine formula (great-circle distance on Earth's surface) and the list is sorted nearest-first.
- ☒ **"Booking" action:** Purrdict doesn't run its own appointment-scheduling backend — instead, tapping a clinic opens a detail view with one-tap  **Call Now** (`tel:` link straight to the clinic's phone) and  **Directions** (opens the address in Google Maps) buttons, so the owner books the visit by phone or navigates there directly.
- ☒ *Current limitation:* there is no `/api/vets/search` route in the codebase at all (the folder for it exists but is empty) — the entire clinic list lives client-side in `health/page.tsx` as a hardcoded `WORLDWIDE_VETS` object covering 8 regions (Philippines, NYC, London, Tokyo, Sydney, Toronto, Berlin, Singapore) with real lat/lng per clinic. The distance math, geolocation, and call/directions actions are fully functional against that static dataset, but there is no live search API behind it despite the README describing Overpass API usage.

Needs & Health Monitor

- ☒ Needs predictor — estimates hunger, thirst, play, rest, attention, and vet-check needs with confidence percentages, sorted by likelihood
- ☒ Health monitor — 7-day baseline chart plus activity metrics (steps, jumps, grooming minutes, feeding events, water visits, restlessness) with a "Detection Methods" citation panel, plus a "Schedule a vet visit?" prompt that opens the Vet Finder above
- ☒ *Current limitation:* both pages are fully client-side with no supporting API route. The Needs Predictor renders a fixed array from `src/lib/mockData.ts` (`needPredictions`) — the numbers never change. The Health Monitor calls `generateHealthWeek()` / `generateSensorMetrics()` , which produce fresh `Math.random()` values on every page load, labeled with sensor names (MPU6050, INP) that don't correspond to anything in the actual ESP32 firmware, which only has a camera and frame-difference motion detection — no accelerometer, gyroscope, or proximity sensor exists in the current hardware code.

Scrapbook — Photo/Video/Note Albums

- ☒ **Create albums:** owners create named "books" with a custom cover color and pattern (dots, stripes, zigzag, stars, hearts) — each book is a separate album, e.g. one per cat or one per season.
- ☒ **Add memories:** inside a book, owners add entries of 3 types — photo, video (both up to 50MB, converted to base64 on upload), or a written note — each with a caption, optional full story text, and an optional mood tag (milestone/funny/chaos/cozy/wholesome).

- ☒ **Book-style browsing:** entries render two-per-spread like an actual photo album, with a page-flip animation and dot-style page indicators; tapping an entry opens a full-size detail view with the media, caption, date, and tag.
- ☒ **Local-first sync:** every create/delete updates the on-screen list and `localStorage` immediately, then syncs to the database in the background via `/api/scrapbook/books` and `/api/scrapbook/entries` — if the DB call fails, the memory still shows up and persists locally instead of disappearing.
- ☒ *Known edge case:* the original `scrapbook_entries` schema (`sql/001_schema.sql`) defines `cat_id` `UUID NOT NULL` , but `POST /api/scrapbook/entries` looks up `cats` `WHERE owner_id = $1 LIMIT 1` and passes `null` if the user has no cat yet. For a real database (not demo mode), saving a scrapbook entry before adding a first cat would fail the `NOT NULL` constraint — the local-first fallback masks this in the UI since the entry still appears from `localStorage` even if the background DB sync throws.

Reminders

- ☒ Categorized tasks (feeding/health/play/grooming/vet/other) with priority levels, recurrence, completion tracking, and a completion-rate ring chart
- ☒ Durable reminder scheduling workflow (`reminderSchedulerWorkflow` via Temporal, triggered through `/api/reminders/schedule`) exists on the backend, though the current Reminders dashboard list runs on local state rather than calling that endpoint yet

Platform, Security & AI-Agent Integration

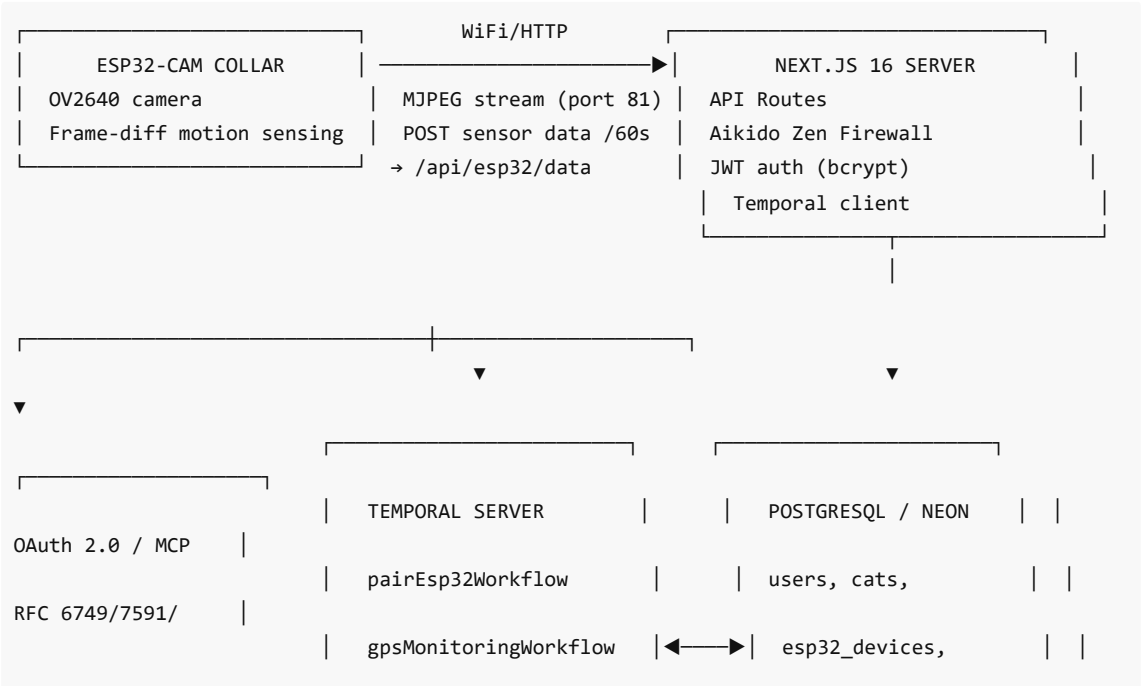
- ☒ 4 Temporal workflows exist in the codebase (`pairEsp32Workflow` , `reminderSchedulerWorkflow` , `gpsMonitoringWorkflow` , `behaviorAnalysisWorkflow`), each with its own activities and retry policy (5 attempts, exponential backoff)
- ☒ Aikido Zen runtime firewall plus CSP/HSTS/X-Frame-Options security headers on every response
- ☒ OAuth 2.0 authorization server (RFC 6749 Authorization Code + PKCE S256, RFC 7591 Dynamic Client Registration, RFC 9728 Protected Resource/Authorization Server metadata)
- ☒ MCP (Model Context Protocol) server exposing 6 tools — list cats, get cat health, get cat location, get scrapbook, get reminders, create reminder — so AI agents like Claude can query a cat's data through an authenticated connector
- ☒ Android app via Capacitor, wrapping the same web dashboard in a native shell
- ☒ *Current limitations, confirmed by reading the source:*
 - **`behaviorAnalysisWorkflow` is never invoked anywhere in the app.** The actual live behavior classification path (`POST /api/esp32/data`) writes directly to Postgres and does not go through Temporal at all — the workflow exists in `src/temporal/workflows.ts` but has no caller.
 - **`pairEsp32Workflow` , `reminderSchedulerWorkflow` , and `gpsMonitoringWorkflow` each have a working API route** (`/api/esp32/pair` , `/api/reminders/schedule` , `/api/gps/monitor`), but no page in the dashboard UI currently calls any of those three routes — the Setup wizard's "pair leash" step and the Map/Reminders pages use their own separate, non-Temporal logic instead.
 - **MCP tool `list_cats` is not scoped to the authenticated user** — it runs `SELECT ... FROM cats ORDER BY created_at LIMIT 50` with no `WHERE owner_id = ...` clause, so an authenticated MCP client can currently read every user's cats, not just their own.
 - **MCP tool `get_reminders` 's `include_done` filter has a logic bug:** the query builds `WHERE done = false ${doneFilter ? "" : "OR true"}` , and since `doneFilter` is always an empty string when `include_done` is false, the condition evaluates to `WHERE done = false OR true` , which is always true — the tool always returns every reminder regardless of the flag.
 - **`GET /api/debug-oauth` is unauthenticated** and returns OAuth client internals (registered `redirect_uris` , client metadata) for any `client_id` passed in the query string — it's marked in its

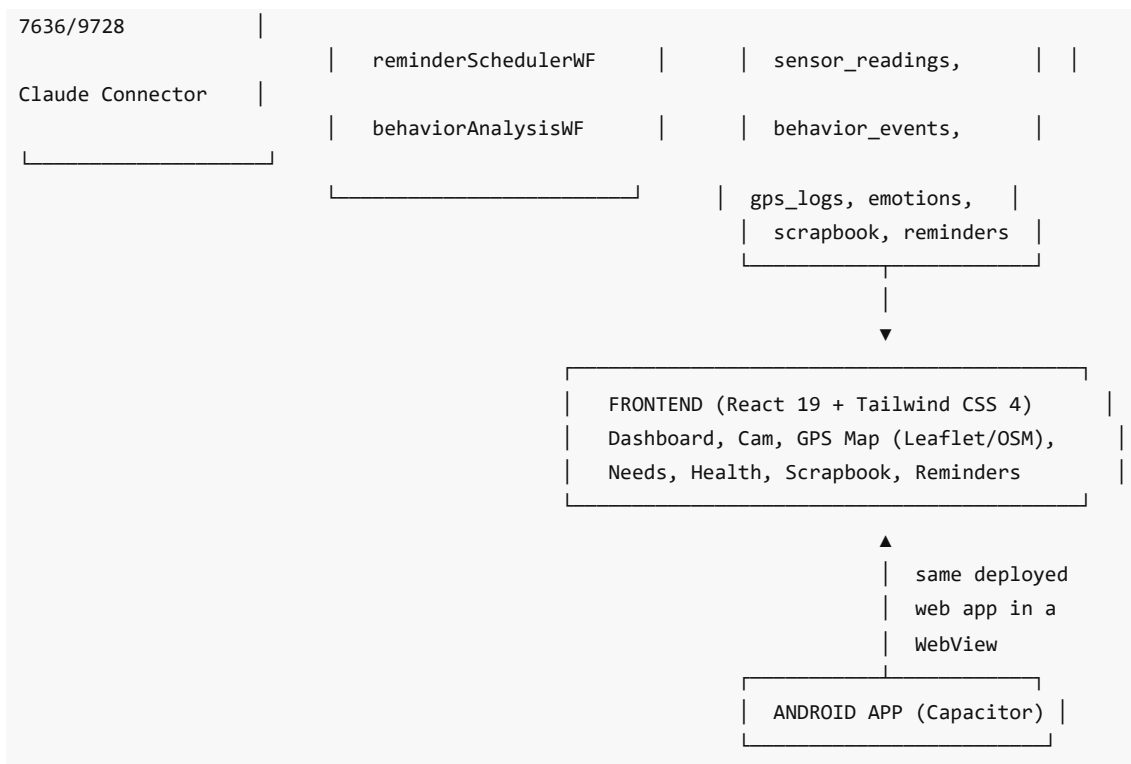
own source comment as a temporary route to delete.

4. Technology Stack

Layer	Technology
Frontend	Next.js 16.2.9, React 19.2.4, Tailwind CSS 4
Mobile	Capacitor 8 (Android), wraps the deployed web app in a WebView
Backend / API	Next.js API routes (Route Handlers)
Database	PostgreSQL / Neon serverless (@neondatabase/serverless), raw parameterized SQL, no ORM
Auth	bcryptjs (cost 12) + jsonwebtoken (custom JWT)
Durable workflows	Temporal TypeScript SDK (@temporalio/*)
Security	Aikido Zen Firewall (@aikidosec/firewall), CSP/HSTS/X-Frame-Options headers
Maps / Geo	Leaflet.js + react-leaflet, OpenStreetMap (CartoDB dark tiles), Nominatim geocoding, browser Geolocation API
AI-agent integration	mcp-handler (MCP server) behind a custom OAuth 2.0 authorization server
Hardware / Firmware	ESP32-CAM (AI-Thinker, OV2640 camera), Arduino/C++ firmware
Deployment	Vercel (purrdict.vercel.app)

5. Technical Architecture





Data flow: The ESP32-CAM captures frames, estimates motion intensity from JPEG frame-size differencing (a proxy for ODBA, per Ikurior et al. 2023), and POSTs that reading to `/api/esp32/data` roughly once a minute. That route looks up or auto-registers the device, classifies behavior from motion-intensity thresholds (e.g. `>80% → running`, `40-60% → walking`, `<5% → resting`), and writes directly to the `sensor_readings` and `behavior_events` tables via a plain Postgres query — this live classification path does **not** go through Temporal. The Cat Cam page separately proxies the ESP32's live MJPEG stream through `/api/esp32/stream` to avoid mixed-content/CORS issues, since the ESP32 only serves plain HTTP. Temporal workflows for device pairing, GPS logging, reminder scheduling, and behavior analysis exist with real retry policies, but each one currently sits behind an API route (`/api/esp32/pair`, `/api/gps/monitor`, `/api/reminders/schedule`) that no dashboard page calls yet — they're built and ready to wire up but aren't in the live request path today. The Android app doesn't contain separate app logic — it's a Capacitor WebView shell (`appId: app.purrdict.cat`) that loads the same deployed site at `purrdict.vercel.app`, so the web dashboard is the single source of truth for all clients. The database layer has an explicit "demo mode": if `DATABASE_URL` isn't set, API routes return safe canned/empty responses instead of failing, which is what powers the one-click Demo Mode experience.

6. Testing Matrix

Feature / Flow	Steps	Expected Result	Actual Result	Pass / Fail
Production build	Run <code>npm run build</code>	Next.js compiles and generates all static + dynamic routes with 0 errors	Verified directly: build completed in ~20s, TypeScript check passed, 32 routes generated (12 static, 8 dynamic API, plus OAuth/well-known routes), exit code 0	Pass

ESP32 behavior classification	POST { motionIntensity: 90 } to /api/esp32/data	Behavior classified as "running" with ~0.93 confidence	Confirmed by reading the route's threshold logic (>80 → running, 0.93)	Pass (verified in code)
ESP32 behavior classification	POST { motionIntensity: 2 } to /api/esp32/data	Behavior classified as "resting" (default), confidence 0.90	Confirmed in code — falls through all thresholds to the default "resting" branch	Pass (verified in code)
Cat card CRUD	Create, edit, delete a cat card	Card persists via /api/cats, updates via PATCH, removes via DELETE	Confirmed in code — real DB-backed CRUD scoped to the authenticated owner	Pass (verified in code)
Scrapbook album + entry	Create a book, add a photo entry	Entry saved via /api/scrapbook/entries, visible on next load; falls back to local-only storage if DB is unreachable	Confirmed in code — optimistic local update + background DB sync with silent fallback	Pass (verified in code)
Live Cat Cam stream	Open Dashboard → Cam with a reachable ESP32 IP configured	MJPEG stream renders via /api/esp32/stream proxy, LIVE badge shown	Not run against physical hardware during this review — needs a live ESP32 on the same network to confirm end-to-end	Not yet verified
GPS map	Open Dashboard → Map	Cat marker moves along a trail inside the geofence	Confirmed in code that the marker path is a simulated fixed loop around the home address, not live GPS from hardware yet — map rendering and geofence math are real, movement data is a demo stand-in	Pass (as a demo), flagged as simulated
GPS distance-from-home + away alert	Let the simulated path move the cat past 80m from home	Distance readout updates in meters, badge flips from "● SAFE ZONE" to "⚠ OUTSIDE" once past the geofence radius	Confirmed in code — distance is computed live via an equirectangular lat/lng-to-meters formula and compared against GEOFENCE_RADIUS = 80; badge and progress bar color both flip correctly at the threshold	Pass (verified in code)
Vet Finder distance ranking	Set a home address, search a city	Clinic list sorts nearest-first with distance shown per clinic	Confirmed in code — haversine formula computes km/mi distance from home to each clinic's stored lat/lng and	Pass (verified in code)

			sorts ascending before rendering	
Vet Finder booking action	Open a clinic's detail view	Call and Directions buttons work	Confirmed in code — "Call Now" is a tel: link to the clinic's phone number, "Directions" opens Google Maps with the clinic's address; no in-app appointment scheduling exists, by design	Pass (verified in code)
Reminders list	Add / complete / delete a reminder	Change reflects immediately and persists after refresh	Confirmed in code — reminders currently live only in in-memory React state; the change reflects immediately but does not persist after a page refresh , even though a real backend/Temporal workflow exists for reminders elsewhere in the app	Fail — UI not yet wired to the persistence layer
Needs predictor / Health monitor	Open Dashboard → Needs / Health	Metrics reflect actual sensor history	Confirmed in code — Needs Predictor renders a fixed sample dataset, and Health Monitor generates random values on every page load; neither reads from the real sensor_readings/behavior_events tables yet	Pass (as a demo), flagged as simulated
Device pairing	POST 6-char alphanumeric PIN to /api/esp32/pair	Starts pairEsp32Workflow via Temporal, returns deviceId on success	Input validation (length, alphanumeric regex) confirmed in code; requires a running Temporal server to exercise end-to-end	Not yet verified
MCP tool list_cats scoping	Call list_cats as an authenticated MCP client	Returns only the calling user's cats	Confirmed in code — the query has no owner_id filter at all, so it returns up to 50 cats across all users	Fail — cross-user data exposure
MCP tool get_reminders filter	Call get_reminders with include_done: false	Returns only reminders where done = false	Confirmed in code — the WHERE done = false \${doneFilter ? "" : "OR true"} clause always evaluates to OR true, so completed reminders are returned regardless of the flag	Fail — filter logic bug
/api/debug-oauth exposure	GET /api/debug-oauth?client_id=<any> with no auth header	Request is rejected or requires authentication	Confirmed in code — the route has no auth check and returns OAuth client metadata for any client_id, as-is, to any caller	Fail — should be removed or gated

				before submission
--	--	--	--	-------------------

Note: "flagged as simulated" rows are working, demo-safe features (they render correctly and won't crash), but the underlying data isn't yet coming from live hardware/database — see Future Improvements for the plan to close each gap.

8. Future Improvements

Bugs to fix first (found during this code review):

- ☐ Scope the MCP `list_cats` tool query to the authenticated user's `owner_id` — it currently has no `WHERE` clause and can return other users' cats
- ☐ Fix the `get_reminders` MCP tool's `include_done` filter — the current `WHERE done = false` `${doneFilter ? "" : "OR true"}` clause always matches every row
- ☐ Remove or authenticate `GET /api/debug-oauth` before submission — it exposes OAuth client metadata with no access control
- ☐ Handle the case where a scrapbook entry is created before the user has any cat — `cat_id` is `NOT NULL` in the schema but the entries route can pass `null`

Wire up what's already built:

- ☐ Connect the Setup wizard's "pair leash" step, the Reminders page, and the Map page to their existing Temporal-backed endpoints (`/api/esp32/pair` , `/api/reminders/schedule` , `/api/gps/monitor`) — all three workflows exist and work, but nothing in the UI currently calls them
- ☐ Route live behavior classification through `behaviorAnalysisWorkflow` instead of writing directly to Postgres from `/api/esp32/data` , so auto-clip-on-high-confidence and durable retries actually take effect
- ☐ Wire the Reminders dashboard UI to `/api/reminders/schedule` and the `reminders` table so tasks persist across sessions instead of resetting on refresh

Replace simulated data with real sensors:

- ☐ Add an actual accelerometer/gyroscope (e.g. MPU6050) and GPS module to the ESP32 firmware — the current hardware only has a camera and does frame-size motion differencing; there is no IMU or GPS chip in the firmware today despite the Setup wizard and Health page referencing them
- ☐ Replace Health Monitor's `Math.random()` metrics and the Needs Predictor's static `mockData.ts` array with real aggregates from `sensor_readings` / `behavior_events`
- ☐ Replace the GPS map's and `gpsMonitoringWorkflow` 's simulated coordinate generation with real ESP32 GPS readings, once that hardware exists
- ☐ Implement a real emotion-assessment pipeline that writes to `emotion_assessments` (table and Temporal activity already exist) instead of the current client-side lookup keyed off the currently-shown behavior label
- ☐ Implement live Overpass API (or Google Places) clinic search for Vet Finder instead of the current curated static clinic list, and add a real `/api/vets/search` route (currently missing entirely)

Polish:

- ☐ Add push notifications for geofence breaches and wellness anomaly alerts instead of requiring the dashboard to be open
- ☐ Persist and surface Aikido Zen scan reports directly in-app for judges/users to review security posture

- ☐ Add automated integration tests around the `/api/esp32/*` routes and Temporal workflows (currently verified only by manual code review and build checks)

9. Tools You Used

- ☒ Kiro (AI coding assistant) — used throughout development per the hackathon's tooling requirement
- ☒ Aikido Zen — runtime application security (also a required hackathon technology)
- ☒ Temporal — durable workflow orchestration (also a required hackathon technology)
- ☐ *[Add any design tools, recording software, or other assistants your team used]*

11. Learnings & Takeaways

Building Purrdict end-to-end in under two weeks (per the team's day-by-day log) surfaced a few concrete lessons: mapping sensor signals to cited research constrained the design in a good way — it forced concrete, falsifiable thresholds (e.g. motion-intensity bands) instead of vague "AI detects X" claims. Integrating Temporal for workflows that must survive restarts (device pairing, GPS monitoring, reminders) required thinking in terms of retries and idempotency rather than fire-and-forget API calls. Wiring a real Leaflet/OpenStreetMap view exposed SSR and hydration pitfalls specific to Next.js that needed a direct-API workaround instead of `react-leaflet`'s declarative components. Building the demo-mode fallback into every API route early (instead of bolting it on later) made it much easier to keep shipping UI before every backend piece (Temporal, real GPS hardware, live sensor feeds) was fully wired up. *[Team: expand with your own reflections — what surprised you, what you'd do differently.]*

12. Acknowledgments

- Research citations: 17 peer-reviewed papers underpin the behavior, emotion, and health features (see full list with DOIs in `README.md`), including Ikurior et al. 2023, Evangelista et al. 2019/2023, and Nicholson & O'Carroll 2021.
- Open-source: Next.js, React, Leaflet, OpenStreetMap/Mapbox tile data, Temporal, Aikido Zen, `mcp-handler`.
- [Add any mentors or hackathon organizers you'd like to credit.]*

Submission Checklist

- ☐ Video demo (HD or at least 720p)
- ☐ README.md (prerequisites, run instructions, configuration) — already present at project root
- ☒ Project report (this document, in `documentation/`)
- ☐ Source code in `src/` — already present at project root
- ☐ No unrelated files, executables, auto-generated code, or package folders (e.g. `node_modules/`, `build/`, `dist/`, `vendor/`) — confirm these are excluded from the submission zip before uploading

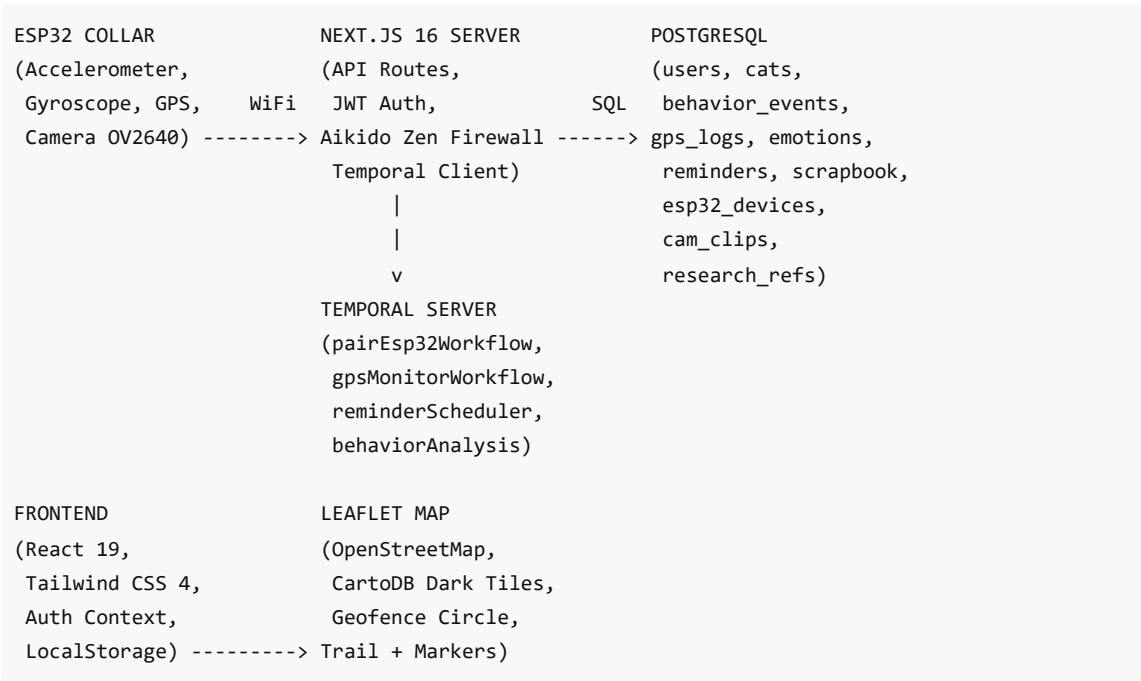
Appendix: Development Progress Log (REPORT.md)

The section below is the team's original day-by-day development log, kept for reference.

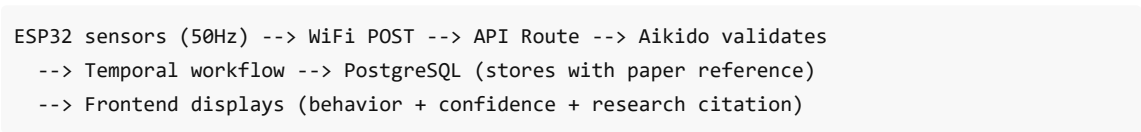
Purrdict - Development Progress Report

Hackathon: #hackthekitty 2026 Period: June 25 - July 7, 2026

System Architecture



Data Flow



Day 1 - June 25 (Wed): Ideation

Attended the hackathon kick-off. Theme: cats. Must use Aikido, Temporal, Kiro. Judging: Technical 25%, Innovation 20%, Theme 15%, Security 15%, UI 15%, Docs 10%.

Brainstormed concepts. Rejected: cat social media (exists), smart toy (limited), litter tracker (no demo). Winner: ESP32 smart collar that monitors behavior using published research methods, not generic AI.

Named it Purrdict. Decided on Next.js 16 + PostgreSQL + Temporal + Aikido + Leaflet + bcrypt/JWT. Set up repo, installed deps, got skeleton running.

Day 2 - June 26 (Thu): Research

Full day reading papers. Found 17 peer-reviewed publications:

BEHAVIOR CLASSIFICATION:

- Ikuriot et al. 2023 - Accelerometer + Random Forest, >86% accuracy <https://www.mdpi.com/1424-8220/23/16/7165>
- Tattersall et al. 2021 - SOM models, >95% Kappa <https://link.springer.com/10.1038/s41598-021-92896-4>
- Mealin et al. 2024 - 28 cats, 8 behaviors quantified <https://pmc.ncbi.nlm.nih.gov/articles/PMC11053832/>
- Uddin et al. 2024 - CNN-LSTM on embedded collar <https://www.mdpi.com/1424-8220/24/23/7436>
- Delgado et al. 2023 - Gyroscope adds 35% accuracy <https://www.mdpi.com/2076-2615/13/1/120>

6. Ladha et al. 2013 - ML on constrained collar tags <https://www.researchgate.net/publication/319605380>

EMOTION AND PAIN: 7. Nicholson & O'Carroll 2021 - 5 feline emotions defined

<https://pmc.ncbi.nlm.nih.gov/articles/PMC7995744/> 8. Evangelista et al. 2019 - Feline Grimace Scale, 91% sensitivity

<https://www.nature.com/articles/s41598-019-55693-8> 9. Evangelista et al. 2023 - Pain ethogram, 10 categories

<https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0292224> 10. Kumpulainen et al. 2024 - Full

domestic cat ethogram <https://www.mdpi.com/2813-9372/1/3/21>

ACTIVITY AND SLEEP: 11. Miyazaki et al. 2020 - Steps, jumps, sleep quality validated

<https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0236795> 12. Piccione et al. 2013 - Crepuscular

pattern (dawn/dusk peaks) <https://pubmed.ncbi.nlm.nih.gov/3843546/> 13. De Saix et al. 2025 - Light affects behavior

+ cortisol <https://pmc.ncbi.nlm.nih.gov/articles/PMC12182317/>

HARDWARE: 14. Chambers et al. 2022 - Open-source collar spatial tracking <https://www.nature.com/articles/s41598-022-22167-3>

15. Zhang et al. 2020 - UWB radar vital signs through fur <https://www.mdpi.com/2076-2615/10/2/205>

16. Nunes et al. 2024 - ECG vest for cat arrhythmia https://link.springer.com/chapter/10.1007/978-3-031-46933-6_38

17. Ding et al. 2025 - Comprehensive wearable sensor review <https://pmc.ncbi.nlm.nih.gov/articles/PMC12300563/>

Mapped each paper to a feature. Defined behavior categories and detection methods.

Day 3 - June 27 (Fri): Design System + Landing Page

Created the visual identity: pixel art aesthetic with Press Start 2P font, warm cream/cocoa/pink/mint palette. Built CSS custom properties, keyframe animations (bob, blink, scan, twinkle, walk), and utility classes (pixel-border, glass-card, pixel-press).

Built the PixelCat SVG component (48x40 pixel art cat).

Built 6 landing page sections: Nav, Hero (with live tracking scene), Levels (how it works), Showcase (cam + map + scrapbook preview), States (emotion ethogram cards with research citations), CTA + footer.

Day 4 - June 28 (Sat): Dashboard Pages

Built the dashboard shell: layout with bottom nav (5 tabs), TopBar with cat info and live badge.

Created pages:

- Live status: behavior detection feed, activity sparkline, breakdown bars
- Cam: ESP32 camera simulation with scanlines, controls (mute/rec/snap)
- Map: indoor room layout with trail and emoji pins
- Needs: prediction cards sorted by likelihood
- Health: 7-day baseline chart, stats table, alert banner

All using mock data shaped to match future API responses.

Day 5 - June 29 (Sun): Auth + Database

Wrote PostgreSQL schema (sql/001_schema.sql): 11 tables including users, cats, esp32_devices, gps_logs, behavior_events, emotion_assessments, cam_clips, scrapbook_entries, reminders, research_references. All with proper indexes, constraints, and auto-update triggers.

Built API routes:

- POST /api/auth/signup - validates, hashes (bcrypt cost 12), returns JWT
- POST /api/auth/login - verifies password, loads cats, returns JWT

- POST /api/cats - create cat profile
- PATCH /api/cats/[id] - update cat
- DELETE /api/cats/[id] - remove cat

Built login + signup pages. Created AuthContext with localStorage persistence and API calls in background.

Day 6 - June 30 (Mon): Temporal + Aikido

Installed Temporal SDK. Created 4 workflows:

- pairEsp32Workflow: verify PIN, claim device, retry 5x with backoff
- reminderSchedulerWorkflow: check due, mark done, create next recurrence
- gpsMonitoringWorkflow: log coords every 10s, check geofence, run for hours
- behaviorAnalysisWorkflow: classify, log, auto-clip if confidence >80%

Installed Aikido Zen Firewall via instrumentation.ts hook. Added security headers: CSP, HSTS, X-Frame-Options, X-XSS-Protection, Permissions-Policy. All queries parameterized. PIN validation regex.

Day 7 - July 1 (Tue): Research Integration

Replaced generic mock data with research-grounded detections. Each behavior now shows: sensor signature description, confidence %, paper citation, method.

Built Feline Grimace Scale panel (5 action units from Evangelista 2019). Built emotion assessment using Nicholson 2021 (5 emotions + indicators). Built circadian rhythm chart based on Piccione 2013 (crepuscular peaks). Built scrapbook page (book-style with spine, pages, polaroids).

Day 8 - July 2 (Wed): Real Map + Reminders

Replaced fake grid map with real Leaflet/OpenStreetMap. Dark CartoDB tiles, geofence circle (dashed green, 100m), trail polyline (yellow dashed), emoji markers with popups, pulsing current-position dot.

Fixed Leaflet SSR issue by using direct API import instead of react-leaflet. Added invalidateSize() workaround for tile loading after React hydration.

Built reminders page: categories (feeding/health/play/grooming/vet/other), priorities (high/med/low), recurring support, completion tracking.

Day 9 - July 3 (Thu): Onboarding + Cat Cards

Built 4-step setup wizard: PIN entry -> name cat -> preview card -> done. PIN input: 6 boxes, auto-focus, backspace navigation, connection simulation. Cat profile: SVG icon picker, photo upload, breed, age, fur color swatches. Card preview: trading card style with color band, stats grid, PIN display.

Built CatCardList on dashboard with EDIT modal and DELETE functionality. Fixed critical bug: addCat failed silently when user was null. Fixed by creating guest user on the fly if needed.

Day 10 - July 4 (Fri): Landing Redesign

Redesigned Hero: dark live-tracking scene with SVG path, glass pin badges, pulsing cat marker, status bar.

Split Showcase into two sections: Spy Cam (full-width cam + map below) and Scrapbook (book-style preview with dot paper, polaroids, spine).

Updated nav with Login/Signup buttons. Added backdrop blur.

Day 11 - July 5 (Sat): Sign Out + Observation Feed

Added sign out button to TopBar. Clears state + localStorage + JWT.

Overhauled dashboard: now shows ESP32 collar observations as a live feed. Each detection has: behavior name, sensor description (e.g. 'Rhythmic Y-axis pattern 0.8-1.2Hz'), confidence bar, research method citation.

Simulates new observations every 30s. 10 behavior types with detailed accelerometer descriptions from the papers.

Day 12 - July 6 (Sun): Polish + Fixes

Fixed GPS map rendering (tiles only half-loaded). Rewrote GpsMap component using direct Leaflet API with proper mount timing.

Increased all UI sizes for desktop (was too small at max-w-md). Layout -> max-w-2xl, bigger fonts, taller bars, more padding throughout.

Updated landing page: removed all AI language, replaced with research citations. Levels now cite specific papers. States section shows the actual ethogram with paper links. Footer: 'PURRDICT RESEARCH' not 'AI'.

Day 13 - July 7 (Mon): Documentation + Submission

Wrote README.md: architecture, setup guide, 17 paper citations with DOIs, security documentation, Temporal workflow descriptions.

Wrote this progress report.

Final build: 0 errors, 20 routes (12 static + 8 dynamic API).

Summary

- 17 research papers integrated
- 8 API routes
- 4 Temporal workflows
- 11 database tables
- 7 dashboard pages
- Real OpenStreetMap GPS tracking
- Aikido Zen Firewall runtime protection
- Full auth flow with cat card CRUD
- 0 build errors